familiar with, for example, TensorFlow, SPARK, or any kind of basic language they can start with—at least C or C++. We compile it into some intermediate representation internally for which we have HeteroCL[FPGA'19]/MLIR.

MLIR is a community-wide effort that was originally started by Google and Intel. They are working to create automatic customised architectures, such as Systolic arrays and Stencil computation, and there is also a new type of architecture with product compostable parallel and pipeline (CPP).

If you have a certain pattern then we can generate this automatically for you. If not, we can use machine learning techniques to look for the best optimisation. As a programmer, you just have to write an efficient CPU code.

Our manual systolic array design experience has been decent so far. We started this effort back in 2014 without knowing what Google had been doing.

We independently designed a systematic way for matrix multiplication with the help of a very bright undergraduate student. It was a manually designed 1D systolic array with over 1700 lines of RTL code. We did get a running close to 200 GFLOPs on Xilinx VC709.

A closer look at computation management throws up the following:

Space-time mapping. Transforms the program to a systolic array with space-time mapping.

Array partitioning. It partitions the array into smaller sub-arrays to fit limited on-chip resources.

Latency hiding. Permutes parallel loops inside to hide computation latency.

SIMD vectorisation. Vectorises computation to amortise the PE overheads.

We have adopted the Merlin compiler, which was developed by Falcon Computing. It is similar to OpenMP for parallel computation, with which many of you are already familiar.

Overview of AutoSA compilation flow

Today, we have a completely automated tool. It extracts the polyhedral model from the source code and examines if the target program can be mapped to the systolic array. It constructs and optimises PE arrays (space-time mapping, array partitioning, latency hiding, vectorisation) and I/O network (I/O network analysis, double buffering, data-packing). It also generates target hardware code.

You can write programs like #pragma ACCEL parallel, which runs multiple loop iterations in parallel (instruction/task-level). Inside the compiler you can automate transformation. It allows you to perform on-chip memory banking/partitioning/delinearisation, external memory bursting/streaming/coalescing.

Current goal: Zero pragma

We need to build things on top of what we already have. For example, if you want to do double Matrix vector multiplication, and you have some good hardware design tuition, then these pragmas are reasonable, and this is what we want to remove completely. (A pragma is a compiler directive that allows you to provide additional information to the compiler.)

We are going to do this using various machine learning algorithms. Along with this we are also going to use several deep learning models. This will be done in following steps:

Step 1: Create a database for training the model. We are going to take the C++ code and run it in the Merlin compiler. We will be getting a solution, which we will analyse. This will help us find the bottleneck, which will allow us to pipeline the rows. We essentially train a machine learning agent and we do it for a whole bunch of applications.

Step 2: Represent the program as a graph. We need to build the graph using the LLVM IR to capture lower-level instructions. We need to include both the program semantic and pragma flow in the graph (program semantic: control, data, and call flow). The graph is generated once per kernel and filled with different pragma values later on.

Step 3: Build a predictive model. From here we are going to create a graph of neural networks. We will first do the encoding here. Each node consists of a feature and you are going to compute a new feature iteratively based on your neighbours. This matrix can also be learned. It is similar to deep learning for images. Eventually, after adding them all together, we will get a graphing value that represents the program.

Other efforts to make FPGAs easier to program

- We can support legacy code (for example, with pointers and recursions): HeteroRefactor.
- We can support task level parallelism: TAPA.
- We can do performance debugging.
- We can do better integration for near-storage acceleration: Insider.
- We can improve the clock frequency for HLS design: AutoBridge.

This is a golden age for computer architecture, and we want to support every programmer who wants to participate and wants to build their own customised accelerators on field-programmable fabrics. And we hope that many people will be on board with us for the same. EFY

This article is based on a tech talk session at VLSI 2022 by Jason Cong, Volgenau Chair for Engineering Excellence, and Director of VAST, UCLA. It has been transcribed and curated by Laveesh Kocher, a tech enthusiast at EFY with a knack for open source exploration and research.